Lecture 5: Types

COMP2221: Functional programming

Maximilien Gadouleau

m.r.gadouleau@durham.ac.uk

Important types in Haskell

Some important types

We have already encountered some types in our lectures, such as:

- `Int`: Bounded precision integers
- `Integer`: Arbitrary precision integers
- `Float`: Single-precision floating point numbers
- `Double`: Double-precision floating point numbers
- `Char`: Unicode characters
- `Bool`: Booleans, with values `True` and `false`, conjunction `(&&)`, disjunction `(||)`, and negation `not`, where `otherwise == True`
- Lists: any sequence of elements of the same type, e.g. `[1, 4, 5]` or `['b', 'd']`
- Tuples: anything in brackets, of the form `( x1, x2 )` or `( x1, x2, x3 )` and so on, with support to `curry` and `uncurry` functions

Adding new data types

Defining data types

- It often makes sense to *define* new data types
- Multiple reasons to do this:
  1. Hide complexity
  2. Build new abstractions
  3. Type safety
- Haskell has three ways to do this
  - `type`
  - `data`
  - `newtype`

Type declarations: new names, old types

- A new *name* for an existing type can be defined using a *type declaration*

String as a synonym for the type [Char]

```
type String = [Char]

vowels :: String -> [Char]
vowels str = [s | s <- str, s `elem` ['a', 'e', 'i', 'o', 'u']]

Prelude> vowels "word"
"o"
Prelude> vowels ['w', 'o', 'r', 'd']
"o"
```

- Notice that there is no type distinction: objects of type `String` and `[Char]` are completely interchangeable.

New names, old types II

- We can use these type declarations to make the semantics of our code clearer

An integer position in 2D

```haskell
type Pos = (Int, Int)

origin :: Pos
origin = (0, 0)

left :: Pos -> Pos
left (i, j) = (i - 1, j)
```

- Reader has to expend less brain power to understand the function
- Similar to C's `typedef`

New names, old types III

- Just like function definitions, type declarations can be parameterised over *type variables*

Example

```
type Pair a = (a, a)

mult :: Pair Int -> Int
mult (m, n) = m*n

dup :: a -> Pair a
dup x = (x, x)
```

✗ Can't use *class constraints* in the definition

✗ Can't have *recursive* types

Not allowed

```
Prelude> type Tree = (Int, [Tree])
error:
    Cycle in type synonym declarations:
```

Data declarations: new types

- We can introduce a completely *new* type by specifying allowed values using a *data declaration*

A boolean type

```
data Bool = False | True
```

"Bool is a new type, with two new values: False, and True"

- The two values are called *constructors* for the type `Bool`
- Both the type name, and the constructor names, must begin with an upper-case letter.
- This is actually the way `Bool` is implemented in the standard library

Using new types

- Once defined, we can use new types exactly like built in ones

Example

```haskell
data IsTrue = Yes | No | Perhaps

negate :: IsTrue -> IsTrue
-- Pattern matching on constructors
negate Yes     = No
negate No      = Yes
negate Perhaps = Perhaps

Prelude> negate Perhaps
Perhaps
```

Data declarations with fixed type parameters

- The constructors in a data declaration can take arbitrarily many parameters

Example

```
data Shape = Circle Float | Rectangle Float Float
```

"A shape is either a Circle, or a Rectangle. The Circle is defined by one number, the Rectangle by two"

Pattern matching on the constructors:

```
area :: Shape -> Float
area (Circle r) = pi * r^2
area (Rectangle x y) = x * y
```

Data declarations with type variables

- We can also make our data declarations *polymorphic* with appropriate type variables

Example

```haskell
data Maybe a = Nothing | Just a
```

"A `Maybe` is either `Nothing` or else a `Just` with a value of arbitrary type"

```haskell
safehead :: [a] -> Maybe a
safehead [] = Nothing
safehead (x:_) = Just x
```

Recursive types

- Data declarations can *refer to themselves*

Peano numbers

```haskell
data Nat = Zero | Succ Nat
```

"Nat is a new type with constructors `Zero :: Nat` and `Succ :: Nat -> Nat`"

- This type contains the infinite sequence of values

```haskell
Zero
Succ Zero
Succ (Succ Zero)
...
```

- We could use this to implement a representation of the natural numbers, and arithmetic

```haskell
add :: Nat -> Nat -> Nat
add Zero n = n
add (Succ m) n = Succ (add m n)
```

Recursive types II

- This kind of recursive type allows very succint definitions of data structures

Linked list

```
data List a = Empty | Cons a (List a)
intList = Cons 1 (Cons 2 (Cons 3 Empty))
== [1, 2, 3]
```

“A List is either Empty, or a Cons of a value and a List”

Linked list in C

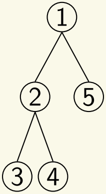
```
typedef struct _Link *Link;
struct _Link {
  void *data;
  Link next;
}
```

A binary tree

A binary tree with values at nodes

```
data BTree a = Empty | Node a (BTree a) (BTree a)
btree = Node 1 (Node 2 (Node 3 Empty Empty)
                       (Node 4 Empty Empty))
               (Node 5 Empty Empty)
```

"A BTree is either Empty, or a Node containing a value and two BTrees"



Pattern matching

- Recall the pattern matching syntax on lists

```
list = [1, 2, 3, 4] == 1:[2, 3, 4]
-- Binds tip to 1, rest to [2, 3, 4]
(tip:rest) = list
```

- The pattern matches the "constructor" of the list, as if the declaration were

```
data [] a = [] | a : [a]
```

- Exactly the same pattern matching applies to data types on their data constructors

```
data List a = Empty | Cons a (List a)
list = Cons 1 (Cons 2 (Cons 3 Empty))
-- Binds tip to 1, rest to (Cons 2 (Cons 3 Empty))
(Cons tip rest) = list
```

Aside: One last way of defining new types

- Haskell also offers to define a new type via `newtype`
- It is very similar to `data`, but only allows for a single constructor, so one can write

  ```
  newtype Rectangle = Rectangle (Int, Int)
  -- allowed
  ```

  but not

  ```
  newtype Maybe a = Just a | Nothing
  -- not allowed
  ```

- In fact, even `newtype Rectangle Int Int` is not allowed either
- By using `newtype`, the constructor is erased at compile time. This can lead to some slight optimisations
- We will stick to `type` and `data` in this course

Some type theory and contrasts

- Haskell's `data` declarations make *Algebraic data types*
- This is a type where we specify the "shape" of each element
- The two algebraic operations are "sum" and "product"

Definition (Sum type)

An alternation:

```
data Foo = A | B
```

A value of type `Foo` can either be `A` or `B`

Definition (Product type)

A combination:

```
data Pair = P Int Double
```

a pair of numbers, an `Int` and `Double` together.

Other languages: product types

- Almost all languages have *product types*. They're just "ordered bags" of things.
- In Python, we can use tuples or classes

Python

```python
pair = (1, 2)
x, y = pair
```

- In C we use structs

C struct

```c
struct Pair {
  int x;
  int y;
}
```

```c
struct Pair p;
p.x = 1;
p.y = 2;
```

- In Java, classes

Other languages: sum types

- Useful for type safety/compiler warnings: easy to statically prove that every option is handled
- Less common, although new languages are catching on (e.g. Rust, Swift)
- In C and Java for integers, you can use an `enum`

```
enum Weekdays {
 MON, TUE, WED, THU, FRI, SAT, SUN
};
```

Haskell types: pros and cons

Classes

- ✓ Easy to add new "kinds of things": just make a subclass
- ✗ Hard to add new "operation on existing things": need to change superclass to add new method and potentially update all subclasses

Algebraic data types

- ✗ Hard to add new "kinds of things": need to add new constructor and update all functions that use the data type
- ✓ Easy to add new "operation on existing things": just write a new function

Pros and Cons II

Adding new things

Just implement a new subclass

```python
class Car(object):
    def seats(self): return 4
class MX5(Car):
    def seats(self): return 2
# Later
class Mini(Car): pass
```

Have to update data constructor

```haskell
data Car = MX5
-- Later
data Car = MX5 | Mini
```

Adding new operations

Must update all classes

```python
class Car(object):
    def mpg(self): return 25
    def seats(self): return 4
class MX5(Car):
    def mpg(self): return 30
    def seats(self): return 2
class Mini(Car):
    def mpg(self): return 40
```

Just write new functions

```haskell
seats :: Car -> Int
seats MX5 = 2
seats Mini = 4
mpg :: Car -> Int
mpg MX5 = 30
mpg Mini = 40
```